Puppet Dnsmasq Module

RHEL 8 / 9 / 10 - custom OpenVox/Puppet module documentation

Purpose: Install dnsmasq and manage its main and project-specific configuration on RHEL 8, 9, and 10.

Module layout

```
custom/
  manifests/dnsmasq.pp
  files/dnsmasq/dnsmasq.conf
  files/dnsmasq/dnsmasq.d/custom.conf
  files/dnsmasq/dnsmasq.d/custom.conf.example
  manifests/rpm_install.pp
  data/common.yaml
```

Configuration

```
include custom::dnsmasq

# Set the trusted certificate extension pp_project to your project.
# Copy files/dnsmasq/dnsmasq.d/custom.conf.example to
# files/dnsmasq/dnsmasq.d/custom.conf.<pp_project>.
# Example project setting:
server=/project.example/192.0.2.53

# Keep dnsmasq out of custom::rpm_install::xrpms.
```

Behavior and safety: The trusted pp_project extension selects custom.conf.<pp_project>, falling back to custom.conf when no matching file exists or no project is set. Invalid project identifiers fail catalog compilation. Puppet includes rpm_install, avoids duplicate package ownership, and rejects dnsmasq in xrpms. Configuration files are root-owned with mode 0644; changes notify the enabled, running dnsmasq service. The shipped default project configuration contains comments only.

Dnsmasq

custom/manifests/dnsmasq.pp

```
# @summary
#   Installs dnsmasq and selects configuration by the trusted project extension.
class custom::dnsmasq {
  if $facts['os']['family'] != 'RedHat' or $facts['os']['release']['major'] !~ /^(8|9|10)$/ {
    fail('custom::dnsmasq supports only RHEL 8, RHEL 9, and RHEL 10')
  }

  $project = $trusted['extensions']['pp_project']
  if $project == undef or $project == '' {
    $project_sources = []
  } else {
    if $project !~ /^[A-Za-z0-9][A-Za-z0-9_.-]*$/ {
      fail('custom::dnsmasq pp_project must contain only letters, digits, underscores, dots, and hyphens and start with a letter or digit')
    }
    $project_sources = ["puppet:///modules/custom/dnsmasq/dnsmasq.d/custom.conf.${project}"]
  }

  include custom::rpm_install

  if 'dnsmasq' in $custom::rpm_install::xrpms {
    fail('custom::dnsmasq cannot be used when dnsmasq is in custom::rpm_install::xrpms')
  }

  # Including rpm_install first makes package ownership independent of class order.
  if !('dnsmasq' in $custom::rpm_install::rpms) {
    package { 'dnsmasq':
      ensure => installed,
    }
  }

  file { 'custom dnsmasq configuration directory':
    ensure  => directory,
    path    => '/etc/dnsmasq.d',
    owner   => 'root',
    group   => 'root',
    mode    => '0755',
    require => Package['dnsmasq'],
  }

  file { 'custom /etc/dnsmasq.conf':
    ensure  => file,
    path    => '/etc/dnsmasq.conf',
    owner   => 'root',
    group   => 'root',
    mode    => '0644',
    source  => 'puppet:///modules/custom/dnsmasq/dnsmasq.conf',
    require => Package['dnsmasq'],
    notify  => Service['dnsmasq'],
  }

  file { 'custom /etc/dnsmasq.d/custom.conf':
    ensure       => file,
    path         => '/etc/dnsmasq.d/custom.conf',
    owner        => 'root',
    group        => 'root',
    mode         => '0644',
    source       => $project_sources + [
      'puppet:///modules/custom/dnsmasq/dnsmasq.d/custom.conf',
    ],
    sourceselect => 'first',
    require      => File['custom dnsmasq configuration directory'],
    notify       => Service['dnsmasq'],
  }
```

```
  service { 'dnsmasq':
    ensure  => running,
    enable  => true,
    require => [
      Package['dnsmasq'],
      File['custom /etc/dnsmasq.conf'],
      File['custom /etc/dnsmasq.d/custom.conf'],
    ],
  }
}
```

Dnsmasq

custom/files/dnsmasq/dnsmasq.conf

```
# Managed by Puppet. Put project-specific settings in dnsmasq.d/custom.conf.
conf-dir=/etc/dnsmasq.d,*.conf
```

```
# Managed by Puppet. Default when no project-specific configuration exists.
# Add shared default settings here.
```

```
# Copy this file to custom.conf.<pp_project> and configure it for that project.
# For example, forward queries for a project domain to its DNS server:
# server=/project.example/192.0.2.53
```

```
# @summary
#   Installs RPM packages supplied through Hiera.
#
# @param rpms
#   Package names to install. Hiera merges this array uniquely across all
#   hierarchy levels.
# @param xrpms
#   Package names to remove. Hiera merges this array uniquely across all
#   hierarchy levels.
class custom::rpm_install (
  Array[String[1]] $rpms = [],
  Array[String[1]] $xrpms = [],
) {
  if $facts['os']['family'] != 'RedHat' {
    fail('custom::rpm_install supports only Red Hat family operating systems')
  }

  $conflicting_rpms = $rpms.filter |String[1] $rpm| {
    $rpm in $xrpms
  }
  if $conflicting_rpms != [] {
    fail("custom::rpm_install cannot install and remove the same packages: ${conflicting_rpms.join(', ')}")
  }

  $rpms.each |String[1] $rpm| {
    package { $rpm:
      ensure => installed,
    }
  }

  $xrpms.each |String[1] $rpm| {
    package { $rpm:
      ensure => absent,
    }
  }
}
```

```
---
lookup_options:
  custom::rpm_install::rpms:
    merge:
      strategy: unique
  custom::rpm_install::xrpms:
    merge:
      strategy: unique

custom::rpm_install::rpms: []
custom::rpm_install::xrpms: []
```